Department of Mechanical, Materials and Manufacturing Engineering

Computer Engineering and Mechatronics

(MMME3085 UNUK) (AUT1 24-25)

Project Planning Report

Author: George Downing (20273662)

Thursday 14th November, 2024

1 Outline of the Problem to be Solved

The program's aim is to print out text on a robot arm by sending G-code instructions over a serial interface. It uses a font file that defines code for different ASCII characters. The program must ask the user for a text file to print and the height of the text to print.

The project will be developed for a cross-platform experience using Windows (MSYS2), Linux, and Mac. GitHub will be used for version control. Automated workflows on Git Hub will compile and run the code to ensure compatibility across platforms. Documentation will be generated via doxygen in an automated workflow on Git Hub in the gh-pages branch, which will then be published on a static website. The program will be compiled using the GCC compiler, which makes it easy to compile on all platforms.

The first aspect of the project will include parsing the font file. The program will use a structure to hold the code instructions. Ideally, this should use dynamic memory allocation to allow for different and more complex fonts with variable strokes/letters. The file will use the fopen() function from stdio.h to open the file. A loop will iterate through all file lines using the fgets() function. When 999 is found, indicating a new character, the program will allocate memory for the new character and the strokes that follow. All characters dynamically allocated will be mapped with a hash table, with the ASCII value of the character as the key.

An interesting problem is selecting the G-code Information given the ASCII character. A simple approach would be to iterate through the array and find the matching ASCII character. This, however, would have a slow time complexity of O(n). A better approach might be to sort the array so the index of the code array is the same as the ASCII value. This would have a time complexity of O(1). However, this requires either sorting the font file or implementing a sorting algorithm. Alternatively, a hash table could be used with an O(1) time complexity. This would allow nonsorted font files to be used efficiently and is easily extendable for other ASCII standards, including Unicode.

The project's next stage will require the scaling of the code instructions; a simple command line dialog using printf() and scanf() will allow the user to input the font size they want. The program will then iterate through characters and font strokes and scale the x and y strokes. The scale is defined by double scale = (double)/ 18.0f;

Next, the stage will be to input the test.txt file. A simple command line dialog using printf() and scanf() will allow users to input the file they want to print out. The file can be opened with fgets(). The specification requires that each word is read from the file individually and processed before reading the next one. While fscanf() could be used to read each individual word, it misses out on the ability to process punctuation or special characters such as a space or \n. Therefore, the best way of processing will be to use fgetc() to read each character individually, appending it to a buffer. When a special character is found, the buffer is copied to an array, which can be processed.

As each word is pulled from the text file, the program will call a function to assemble the code to send to the robot. Static variables will be utilized to keep track of the cursor position. The function will iterate through the word (array of characters), where it will first look for special characters, such as \n, to update the cursor position(garage return with 5mm line spacing). The ASCII value of the character will be used as the key in a hash function to return a pointer to the code instructions for the character. The function will then iterate through the stokes of the character, sending the code to the robot with the appropriate G0 or G1 command for the pen up or down position. Next, the cursor position will be updated to leave a small space between letters. Before sending a new G-code instruction, the program will wait for the OK\n response from the robot to confirm it has room in the buffer for the next instruction. The robot will be communicated via a serial port using the RS232 library provided.

2 Key Data Items

```
typedef struct stroke_s{
    double x;
    double y;
    bool pen_state;
} stroke_t;

typedef struct fontCharacter_s{
    char ascii_key;
    uint8_t num_strokes;
    stroke_t *strokes;
```

```
} fontCharacter_t;

typedef struct cursor_s {
    double x, y;
}cursor_t;

typedef struct hashNode_s {
    char key;
    fontCharacter_t *character;
    struct hashNode_s *next;
} hashNode_t;

typedef struct fontData_s {
    double scale:
    hashNode_t *table[128]; // Hash table
} fontData_t;
```

Name	Data Type	Rationale
file	FILE	part of stdio.h, used to define a file which can be opened with fopen() and worked on with other functions
scale	double	used to scale the gcode font file and character spacing, double for fine presision.
strokes	stroke_t*	structure to hold a single stroke, x and y are double to preserve high presision, and the pen state is bolean because it has 2 states, on or off. memory will be allocated using malloc, so the data type is a pointer hence acts as array
fontCharacter	fontCharacter_t*	Holds the array of strokes and the metadata for a single character. Because the strokes are dynamically allocated memory, the number of strokes is required to know the length of the array of strokes. A char is one byte and, therefore, can hold all combinations of 128 ASCII codes. num_strokes is assigned as an 8-bit unsigned integer, where negative values are impossible, and more than 255 is unlikely. memory will be allocated using malloc, so the data type is a pointer hence acts as array.
Cursor	static cursor_t	Tracks the current position of the "pen" on the robot's surface for precise placement of letters. double allows for floating point presision. will be declared as static within the generate_gcode function to keep track of the cursor position.
hashTable	hashNode_t*	a pointer to the hash table array, contains the key which is used in the hash function to find the correct FontCharacter pointer. although not neccassary becasue the data ascii font file data is sequencial, a HashNode_s structure pointer is used for collision handling. memory will be allocated using malloc, so the data type is a pointer hence acts as array.
Font Data	fontData_t*	A container for holding the hash table array. also holds the scale factor to be used for determining white space. double to preserve presision.

Table 1: Primary Data Types for the G-code Printing Program

3 Function Declarations

```
int hash_table_insert(fontData_t *hash_table, const char ascii_key, fontCharacter_t *character);
```

Breif: Inserts a font character into the hash table.
Parameters:
`hash_table` – Pointer to the hash table.
`ascii_key` – ASCII key for the font character.
`character` – Pointer to the font character to insert.
Returns:
0 on success, -1 on failure.

```
const fontCharacter_t *hash_table_lookup(const fontData_t *hash_table, const char ascii_key);
```

Breif: Looks up a font character in the hash table by ASCII key.
Parameters:
`hash_table` – Pointer to the hash table.
`ascii_key` – ASCII key of the font character to retrieve.
Returns:
Pointer to the corresponding fontCharacter_t, or `NULL` if not found.

```
void parse_font_file(FontData *font_data, const char *filename);
```

Breif: Parses the font file and populates a hash table with font characters.
Parameters:
`font_data` – Pointer to the font data to populate.
`filename` – Name of the font file to parse.

```
void scale_font_data(FontData *font_data);
```

Breif: Scales the font data by a given factor.
Parameters:
`font_data` – Pointer to the font data to scale.

```
void process_text_file(const FontData *font_data);
```

Breif: Processes a text file and generates G-code for the text.
Parameters:
`font_data` – Pointer to the font data to use, also containing scale for whitespace between characters.

```
void generate_gcode(const FontData *font_data, const char *text);
```

Breif: Generates G-code for a given text string.
Parameters:
`font_data` – Pointer to the font data to use, also containing scale for whitespace between characters.
`text` – Text string (single word) to generate G-code for.

```
void free_font_data(fontData_t *font_data);
```

Breif: Frees the memory allocated for the font data. itterate through all hashNode_t,fontCharacter_t and strokes_t to free memory allocated by malloc
Parameters:
`font_data` – Pointer to the font data to be freed.

4 Testing Information

Function	Test Case	Test Data	Expected Output
parse_font_file	empty filename	filename=""	Error message: "Filename is empty"
parse_font_file	fopen() fails	filename="font.exe"	Error message: "unable to open file"
parse_font_file	memory allocation fails	Simulate malloc failure	Error message: "Memory allocation failed"

parse_font_file	normal case	filename= "SingleStrokeFont.txt"	normal operation: calls hash_table_insert(), 128 times
hash_table_insert	NULL hastable	hash_table=NULL	Error message: "Hash table is NULL"
hash_table_insert	NULL character	character=NULL	Error message: "Character is NULL"
hash_table_insert	memory allocation fails	Simulate malloc failure	Error message: "Memory allocation failed"
hash_table_insert	normal case	ascii_key=0, character=fontCharacter_t	normal operation: inserts fontCharacter into hash table
hash_table_insert	insert multiple characters	insert 'A', 'B', 'C'	normal operation: inserts fontCharacter into hash table
hash_table_insert	confirm correct operation with hash_table_lookup	*font_char = hash_table_lookup(font_data, key)	font_char− >ascii_key == Key
hash_table_lookup	NULL hastable	hash_table=NULL	Error message: "Hash table is NULL"
hash_table_lookup	normal case	ascii_key=0	normal operation: returns fontCharacter from hash table with key 0
hash_table_lookup	look up character that doesnt exist	ascii_key=999	Error message: "Character not found"
scale_font_data	NULL font_data	font_data=NULL	Error message: "Font data is NULL"
scale_font_data	input not between 4-10mm	11	Error message: "Input not between 4-10mm"
scale_font_data	test with 'A' 4mm	999 65 6 0 0 0 6 18 1 12 0 1 3 9 0 9 9 1 18 0 0	999 65 6 0 0 0 1.33 4.00 1 2.67 0 1 0.67 2.00 0 2.00 2.00 1 4.00 0 0
scale_font_data	test with 'A' 10mm	999 65 6 0 0 0 6 18 1 12 0 1 3 9 0 9 9 1 18 0 0	999 65 6 0 0 0 3.33 10.00 1 6.67 0 1 1.67 5.00 0 5.00 5.00 1 10.00 0 0
process_text_file	NULL font_data	font_data=NULL	Error message: "Font data is NULL"
process_text_file	fopen() fails	filename="text.exe"	Error message: "unable to open file"
process_text_file	Test each word read indvidually	"Hello World"	normal operation: calls generate_gcode() twice
generate_gcode	NULL font_data	font_data=NULL	Error message: "Font data is NULL"
generate_gcode	test \n mechanism	text[0] = "\n"	cursor moves to next line with 5mm line spaace
generate_gcode	test normal case	text[0] = "A"	normal operation: calls hash_table_lookup() and sends G-code to robot, waits of ok response after every send.
generate_gcode	4mm test 'A' output	text[0] = "A "	$G0 \quad X0.00 \quad Y0.00; \quad pen\ up$ $G1 \quad X1.33 \quad Y4.00; \quad pen\ down$ $G1 \quad X2.67 \quad Y0.00; \quad pen\ down$ $G0 \quad X0.67 \quad Y2.00; \quad pen\ up$ $G1 \quad X2.00 \quad Y2.00; \quad pen\ down$ $G0 \quad X4.00 \quad Y0.00; \quad pen\ down$ $G0 \quad x0 \quad y0; \quad pen\ up$

generate_gcode	10mm test 'A' output	text[0] = "A "	$G0$ $X0.00$ $Y0.00;$ *pen up* $G1$ $X3.33$ $Y10.00;$ *pen down* $G1$ $X6.67$ $Y0.00;$ *pen down* $G0$ $X1.67$ $Y5.00;$ *pen up* $G1$ $X5.00$ $Y5.00;$ *pen down* $G0$ $X10.00$ $Y0.00;$ *pen down* $G0$ $x0$ $y0;$ *pen up*
generate_gcode	test multiple letters checking for proper character spacing with 4mm spacing	text[0] = "AA "	$G0$ $X0.00$ $Y0.00;$ *pen up* $G1$ $X1.33$ $Y4.00;$ *pen down* $G1$ $X2.67$ $Y0.00;$ *pen down* $G0$ $X0.67$ $Y2.00;$ *pen up* $G1$ $X2.00$ $Y2.00;$ *pen down* $G0$ $X4.00$ $Y0.00;$ *pen down* $G0$ $X4.00$ $Y0.00;$ *pen up* $G1$ $X5.33$ $Y4.00;$ *pen down* $G1$ $X6.67$ $Y0.00;$ *pen down* $G0$ $X4.67$ $Y2.00;$ *pen up* $G1$ $X6.00$ $Y2.00;$ *pen down* $G0$ $X8.00$ $Y0.00;$ *pen up* $G0$ $X12.00$ $Y0.00;$ *pen up* $G0$ $X0$ $Y0;$ *pen up*

main	test normal case	SingleStrokeFont, test.txt: "Hello World", 4mm text height	$G0$ $X0.00Y$ 0.00 $Penup$ $G1$ $X0.00Y$ 4.00 $Pendown$ $G0$ $X2.67Y$ 0.00 $Penup$ $G1$ $X2.67Y$ 4.00 $Pendown$ $G0$ $X0.00Y$ 2.00 $Penup$ $G1$ $X2.67Y$ 2.00 $Pendown$ $G0$ $X4.00Y$ 0.00 $Penup$ $G0$ $X4.00Y$ 1.33 $Penup$ $G1$ $X6.67Y$ 1.56 $Pendown$ $G1$ $X6.00Y$ 2.67 $Pendown$ $G1$ $X4.67Y$ 2.67 $Pendown$ $G1$ $X4.00Y$ 2.00 $Pendown$ $G1$ $X4.00Y$ 0.44 $Pendown$ $G1$ $X4.67Y$ 0.00 $Pendown$ $G1$ $X6.00Y$ 0.00 $Pendown$ $G1$ $X6.67Y$ 0.44 $Pendown$ $G0$ $X8.00Y$ 0.00 $Penup$ $G0$ $X8.67Y$ 0.00 $Penup$ $G1$ $X10.00$ $Y0.00;$ $Pendown$ $G0$ $X9.33Y$ 0.00 $Penup$ $G1$ $X9.33Y$ 4.00 $Pendown$ $G1$ $X8.67Y$ 4.00 $Pendown$ $G0$ $X12.00$ $Y0.00;$ $Penup$ $G0$ $X12.67$ $Y0.00;$ $Penup$ $G1$ $X14.00$ $Y0.00;$ $Pendown$ $G0$ $X13.33$ $Y0.00;$ $Penup$ $G1$ $X13.33$ $Y4.00;$ $Pendown$ $G1$ $X12.67$ $Y4.00;$ $Pendown$ $G0$ $X16.00$ $Y0.00;$ $Penup$ $G0$ $X17.33$ $Y0.00;$ $Penup$ $G1$ $X16.00$ $Y0.44;$ $Pendown$ $G1$ $X16.00$ $Y2.00;$ $Pendown$ $G1$ $X17.33$ $Y2.44;$ $Pendown$ $G1$ $X18.67$ $Y2.00;$ $Pendown$ $G1$ $X18.67$ $Y0.44;$ $Pendown$ $G1$ $X17.33$ $Y0.00;$ $Pendown$ $G0$ $X20.00$ $Y0.00;$ $Penup$ $G0$ $X24.00$ $Y0.00;$ $Penup$ $G0$ $X24.00$ $Y4.00;$ $Penup$ $G1$ $X24.67$ $Y0.00;$ $Pendown$ $G1$ $X25.33$ $Y3.11;$ $Pendown$ $G1$ $X26.00$ $Y0.00;$ $Pendown$ $G1$ $X26.67$ $Y4.00;$ $Pendown$ $G0$ $X28.00$ $Y0.00;$ $Penup$ $G0$ $X29.33$ $Y0.00;$ $Penup$ $G1$ $X28.00$ $Y0.44;$ $Pendown$ $G1$ $X28.00$ $Y2.00;$ $Pendown$ $G1$ $X29.33$ $Y2.44;$ $Pendown$ $G1$ $X30.67$ $Y2.00;$ $Pendown$ $G1$ $X30.67$ $Y0.44;$ $Pendown$ $G1$ $X29.33$ $Y0.00;$ $Pendown$ $G0$ $X32.00$ $Y0.00;$ $Penup$ $G0$ $X32.00$ $Y0.00;$ $Penup$ $G1$ $X32.00$ $Y2.44;$ $Pendown$ $G0$ $X32.00$ $Y1.78;$ $Penup$ $G1$ $X33.33$ $Y2.44;$ $Pendown$ $G1$ $X34.67$ $Y1.78;$ $Pendown$ $G0$ $X36.00$ $Y0.00;$ $Penup$ $G0$ $X36.67$ $Y0.00;$ $Penup$ $G1$ $X38.00$ $Y0.00;$ $Pendown$ $G0$ $X37.33$ $Y0.00;$ $Penup$ $G1$ $X37.33$ $Y4.00;$ $Pendown$ $G1$ $X36.67$ $Y4.00;$ $Pendown$ $G0$ $X40.00$ $Y0.00;$ $Penup$ $G0$ $X42.67$ $Y0.44;$ $Penup$ $G1$ $X41.33$ $Y0.00;$ $Pendown$ $G1$ $X40.00$ $Y0.44;$ $Pendown$ $G1$ $X40.00$ $Y2.00;$ $Pendown$ $G1$ $X41.33$ $Y2.44;$ $Pendown$ $G1$ $X42.67$ $Y2.00;$ $Pendown$ $G0$ $X42.67$ $Y4.00;$ $Penup$ $G1$ $X42.67$ $Y0.00;$ $Pendown$ $G0$ $X44.00$ $Y0.00;$ $Penup$ $G0$ $X44.00$ $Y0.00;$ $Penup$ $G0$ $X0$ $Y0;$ $toorigin$

main	test normal case with carrage return	SingleStrokeFont, test.txt: "Hello \n World", 10mm text height	$G0$ $X0.00$ $Y0.00;$ $Penup$ $G1$ $X0.00$ $Y10.00;$ $Pendown$ $G0$ $X6.67$ $Y0.00;$ $Penup$ $G1$ $X6.67$ $Y10.00;$ $Pendown$ $G0$ $X0.00$ $Y5.00;$ $Penup$ $G1$ $X6.67$ $Y5.00;$ $Pendown$ $G0$ $X10.00$ $Y0.00;$ $Penup$ $G0$ $X10.00$ $Y3.33;$ $Penup$ $G1$ $X16.67$ $Y3.89;$ $Pendown$ $G1$ $X15.00$ $Y6.67;$ $Pendown$ $G1$ $X11.67$ $Y6.67;$ $Pendown$ $G1$ $X10.00$ $Y5.00;$ $Pendown$ $G1$ $X10.00$ $Y1.11;$ $Pendown$ $G1$ $X11.67$ $Y0.00;$ $Pendown$ $G1$ $X15.00$ $Y0.00;$ $Pendown$ $G1$ $X16.67$ $Y1.11;$ $Pendown$ $G0$ $X20.00$ $Y0.00;$ $Penup$ $G0$ $X21.67$ $Y0.00;$ $Penup$ $G1$ $X25.00$ $Y0.00;$ $Pendown$ $G0$ $X23.33$ $Y0.00;$ $Penup$ $G1$ $X23.33$ $Y10.00;$ $Pendown$ $G1$ $X21.67$ $Y10.00;$ $Pendown$ $G0$ $X30.00$ $Y0.00;$ $Penup$ $G0$ $X31.67$ $Y0.00;$ $Penup$ $G1$ $X35.00$ $Y0.00;$ $Pendown$ $G0$ $X33.33$ $Y0.00;$ $Penup$ $G1$ $X33.33$ $Y10.00;$ $Pendown$ $G1$ $X31.67$ $Y10.00;$ $Pendown$ $G0$ $X40.00$ $Y0.00;$ $Penup$ $G0$ $X43.33$ $Y0.00;$ $Penup$ $G1$ $X40.00$ $Y1.11;$ $Pendown$ $G1$ $X40.00$ $Y5.00;$ $Pendown$ $G1$ $X43.33$ $Y6.11;$ $Pendown$ $G1$ $X46.67$ $Y5.00;$ $Pendown$ $G1$ $X46.67$ $Y1.11;$ $Pendown$ $G1$ $X43.33$ $Y0.00;$ $Pendown$ $G0$ $X50.00$ $Y0.00;$ $Penup$ $G0$ $X60.00$ $Y0.00;$ $Penup$ $G0$ $X60.00$ $Y0.00;$ $Penup$ $G0$ $X0.00$ $Y-5.00;$ $Penup$ $G1$ $X1.67$ $Y-15.00;$ $Pendown$ $G1$ $X3.33$ $Y-7.22;$ $Pendown$ $G1$ $X5.00$ $Y-15.00;$ $Pendown$ $G1$ $X6.67$ $Y-5.00;$ $Pendown$ $G0$ $X10.00$ $Y-15.00;$ $Penup$ $G0$ $X13.33$ $Y-15.00;$ $Penup$ $G1$ $X10.00$ $Y-13.89;$ $Pendown$ $G1$ $X10.00$ $Y-10.00;$ $Pendown$ $G1$ $X13.33$ $Y-8.89;$ $Pendown$ $G1$ $X16.67$ $Y-10.00;$ $Pendown$ $G1$ $X16.67$ $Y-13.89;$ $Pendown$ $G1$ $X13.33$ $Y-15.00;$ $Pendown$ $G0$ $X20.00$ $Y-15.00;$ $Penup$ $G0$ $X20.00$ $Y-15.00;$ $Penup$ $G1$ $X20.00$ $Y-8.89;$ $Pendown$ $G0$ $X20.00$ $Y-10.56;$ $Penup$ $G1$ $X23.33$ $Y-8.89;$ $Pendown$ $G1$ $X26.67$ $Y-10.56;$ $Pendown$ $G0$ $X30.00$ $Y-15.00;$ $Penup$ $G0$ $X31.67$ $Y-15.00;$ $Penup$ $G1$ $X35.00$ $Y-15.00;$ $Pendown$ $G0$ $X33.33$ $Y-15.00;$ $Penup$ $G1$ $X33.33$ $Y-5.00;$ $Pendown$ $G1$ $X31.67$ $Y-5.00;$ $Pendown$ $G0$ $X40.00$ $Y-15.00;$ $Penup$ $G0$ $X46.67$ $Y-13.89;$ $Penup$ $G1$ $X43.33$ $Y-15.00;$ $Pendown$ $G1$ $X40.00$ $Y-13.89;$ $Pendown$ $G1$ $X40.00$ $Y-10.00;$ $Pendown$ $G1$ $X43.33$ $Y-8.89;$ $Pendown$ $G1$ $X46.67$ $Y-10.00;$ $Pendown$ $G0$ $X46.67$ $Y-5.00;$ $Penup$ $G1$ $X46.67$ $Y-15.00;$ $Pendown$ $G0$ $X50.00$ $Y-15.00;$ $Penup$ $G0$ $X50.00$ $Y-15.00;$ $Penup$ $G0$ $X0$ $Y0;$ $toorigin$

5 Flowcharts

please find flow charts in the file named RobotFlowchartGRD_ID